

Algoritmos e Estruturas de Dados II

50 Questões de Implementação

C & Java — com gabaritos comentados

Categorias

- Lista Dupla Encadeada: 10 questões
- Árvore Binária de Pesquisa: 5 questões
- Matriz Flexível: 6 questões
- Ordenação: 10 questões
- BSP — Operações: 13 questões
- Análise de Complexidade: 6 questões

Questão 1

Categoria: Lista Dupla Encadeada | Linguagem: C | Dificuldade: Médio

Implemente a função void inserirFim(int x) para uma Lista Duplamente Encadeada Circular com Célula Cabeça em C. A lista é representada apenas pelo ponteiro global CelulaDupla* primeiro. A lista é circular: primeiro->ant aponta para o último nó e o último ->prox aponta para primeiro.

Código para implementar:

```
typedef struct CelulaDupla {  
    int elemento;  
    struct CelulaDupla* prox;  
    struct CelulaDupla* ant;  
} CelulaDupla;  
  
CelulaDupla* primeiro; // cabeca sentinela  
  
void inserirFim(int x) {  
    // IMPLEMENTAR  
}
```

Dica: O último nó é sempre acessível via primeiro->ant. Ajuste 4 ponteiros.

Questão 2

Categoria: Lista Dupla Encadeada | Linguagem: C | Dificuldade: Médio

Implemente void removerInicio() para a lista duplamente encadeada circular com cabeça. Se a lista estiver vazia (só a cabeça), não faça nada.

Código para implementar:

```
void removerInicio() {  
    // IMPLEMENTAR  
}
```

Dica: Lista vazia: primeiro->prox == primeiro. Remova o nó logo após a cabeça e atualize os ponteiros vizinhos.

Questão 3

Categoria: Lista Dupla Encadeada | Linguagem: C | Dificuldade: Difícil

Implemente void inverterLista() que inverte in-place a lista duplamente encadeada circular com cabeça, trocando todos os ponteiros prox e ant de cada nó (incluindo a cabeça).

Código para implementar:

```
void inverterLista() {  
    // IMPLEMENTAR  
}
```

Dica: Percorra todos os nós (incluindo a cabeça) e troque prox<->ant em cada um usando do-while.

Questão 4

Categoria: Lista Dupla Encadeada | Linguagem: Java | Dificuldade: Médio

Implemente o método Java CelulaDupla intercalarReverso(CelulaDupla c1, CelulaDupla c2) que recebe os nós cabeça de duas listas duplamente encadeadas e retorna o nó cabeça de uma nova lista construída intercalando elementos do fim de cada lista alternadamente.

Código para implementar:

```
class CelulaDupla {
    int elemento;
    CelulaDupla prox, ant;
}

CelulaDupla intercalarReverso(CelulaDupla c1,
    CelulaDupla c2) {
    // IMPLEMENTAR
}
```

Dica: Use dois ponteiros partindo do final (cab.ant). Avance cada um em direção à cabeça (p = p.ant).

Questão 5

Categoria: Lista Dupla Encadeada | Linguagem: Java | Dificuldade: Fácil

Implemente CelulaLista maiorPilha() na classe Lista (Lista de Pilhas). Retorna a CelulaLista que aponta para a pilha com maior número de elementos. Em caso de empate, retorne a primeira.

Código para implementar:

```
class Lista { CelulaLista inicio, fim; }
class CelulaLista {
    CelulaPilha topo; CelulaLista prox; }
class CelulaPilha {
    int elemento; CelulaPilha prox; }

CelulaLista maiorPilha() {
    // IMPLEMENTAR
}
```

Dica: Para cada CelulaLista, conte os nós da pilha percorrendo a partir do topo. Guarde a referência da maior.

Questão 6

Categoria: Árvore Binária de Pesquisa | Linguagem: Java | Dificuldade: Médio

Na estrutura de árvore de palavras (No com char letra, No esq/dir, No2 raiz; No2 com String palavra, No2 esq/dir), implemente int contarPalavras(char primeiro, char ultimo) que conta palavras que começam com 'primeiro' e terminam com 'ultimo'.

Código para implementar:

```
class ArvoreArvore { No raiz; }
class No {
    char letra; No esq, dir; No2 raiz; }
class No2 {
```

```
String palavra; No2 esq, dir; }
```

```
int contarPalavras(char primeiro, char ultimo) {
```

```
return contarNo(raiz, primeiro, ultimo);
```

```
}
```

```
int contarNo(No no, char pr, char ul) {
```

```
// IMPLEMENTAR
```

```
}
```

Dica: Percorra todos os nós No. Quando letra == primeiro, percorra o No2 verificando a última letra de cada palavra.

Questão 7

Categoria: Árvore Binária de Pesquisa | Linguagem: Java | Dificuldade: Médio

Implemente `int contarPalavras(String padrao)` que conta palavras na árvore que têm a mesma letra inicial e o mesmo número de caracteres que `padrao`. Ex.: 'PROVA' -> conta palavras com P e 5 letras.

Código para implementar:

```
int contarPalavras(String padrao) {
```

```
// IMPLEMENTAR – use a estrutura No/No2
```

```
}
```

Dica: Busque o nó No com a letra inicial. Depois percorra o No2 verificando comprimento.

Questão 8

Categoria: Árvore Binária de Pesquisa | Linguagem: Java | Dificuldade: Difícil

Implemente `void inserir(String palavra)` da classe `Dicionario`. A BSP é indexada pela primeira letra. Cada `No` tem lista encadeada de `Celula` (palavras) em ordem alfabética. Crie o nó `No` se necessário.

Código para implementar:

```
class Dicionario {
```

```
private No raiz;
```

```
public void inserir(String palavra) {
```

```
// IMPLEMENTAR
```

```
}
```

```
}
```

```
class No {
```

```
char letra; No esq, dir;
```

```
Celula primeiro, ultimo;
```

```
}
```

```
class Celula { String palavra; Celula prox; }
```

Dica: Separe: (1) buscar/criar nó No na BSP e (2) inserir palavra na lista ordenada desse nó.

Questão 9

Categoria: Árvore Binária de Pesquisa | Linguagem: Java | Dificuldade: Difícil

Implemente o TreeSort em Java: dado um array de inteiros, use uma BSP para ordená-lo. Método: public static int[] treeSort(int[] arr).

Código para implementar:

```
public static int[] treeSort(int[] arr) {  
    // IMPLEMENTAR  
}
```

// Classe auxiliar: No com int val, No esq, dir

Dica: Duas etapas: inserir cada elemento na BSP, depois caminhamento em-ordem extrai em ordem crescente.

Questão 10

Categoria: Árvore Binária de Pesquisa | Linguagem: C | Dificuldade: Difícil

Implemente Celula* encontrarRepetidos(No* raiz, CelulaMatriz* inicio) que retorna lista encadeada decrescente com números que aparecem tanto na BSP quanto na matriz flexível.

Código para implementar:

```
typedef struct { int numero;  
    struct Celula* prox; } Celula;  
typedef struct { int numero;  
    CelulaMatriz *prox,*ant,*sup,*inf;  
} CelulaMatriz;  
typedef struct No {  
    int numero; struct No *esq,*dir; } No;
```

```
Celula* encontrarRepetidos(No* raiz,  
    CelulaMatriz* inicio) {  
    // IMPLEMENTAR  
}
```

Dica: Percorra toda a matriz (via inf/prox). Para cada elemento, busque na BSP. Mantenha lista resultado decrescente.

Questão 11

Categoria: Matriz Flexível | Linguagem: C | Dificuldade: Médio

Implemente void removerUltimaColuna(Matriz* matriz) que remove a última coluna de uma matriz flexível, ajustando ponteiros e liberando memória.

Código para implementar:

```
typedef struct Celula {  
    int elemento;  
    struct Celula *inf,*sup,*esq,*dir;  
} Celula;  
typedef struct {  
    int linhas, colunas; Celula* inicio;  
} Matriz;
```

```
void removerUltimaColuna(Matriz* matriz) {  
    // IMPLEMENTAR  
}
```

Dica: Percorra linha por linha (via inf). Em cada linha vá até o último nó (dir==NULL), ajuste esq->dir=NULL e libere.

Questão 12

Categoria: Matriz Flexível | Linguagem: C | Dificuldade: Difícil

Implemente void inserirLinha(Matriz* mat, int* valores) que insere uma nova linha no fim da matriz flexível, criando e conectando células com todos os ponteiros (sup, inf, esq, dir).

Código para implementar:

```
void inserirLinha(Matriz* mat, int* valores) {  
    // valores tem mat->colunas inteiros  
    // IMPLEMENTAR  
}
```

Dica: Crie células ligando esq/dir. Para cada célula, conecte sup ao último elemento da coluna correspondente.

Questão 13

Categoria: Matriz Flexível | Linguagem: Java | Dificuldade: Médio

Na Matriz de Listas em Java, implemente void removerImpares() que remove todos os números ímpares de todas as listas internas (CelulaMat com lista de Celula).

Código para implementar:

```
class Matriz { CelulaMat inicio; }  
class CelulaMat {  
    CelulaMat prox, ant, sup, inf;  
    Celula primeiro, ultimo;  
    public CelulaMat(){  
        primeiro = ultimo = new Celula(); }  
}  
class Celula {  
    int numero; Celula prox; }  
  
void removerImpares() {  
    // IMPLEMENTAR  
}
```

Dica: Percorra a matriz (inf/prox). Em cada CelulaMat, percorra a lista interna com ponteiro anterior para remover nós ímpares.

Questão 14

Categoria: Matriz Flexível | Linguagem: Java | Dificuldade: Difícil

Implemente void removerPrimeiraColunaMatriz() que remove a primeira coluna de uma Matriz de Listas, ajustando os ponteiros ant/prox de cada CelulaMat restante.

Código para implementar:

```
void removerPrimeiraColunaMatriz() {
```

```
// IMPLEMENTAR
```

```
}
```

Dica: Percorra a primeira coluna (via inf). Para cada célula, pegue cur.prox e sete segunda.ant = null. Atualize início.

Questão 15

Categoria: Matriz Flexível | Linguagem: C | Dificuldade: Médio

Implemente `Celula* buscaElemento(Matriz* mat, int val)` que retorna ponteiro para a primeira célula com o valor buscado, ou NULL se não encontrado.

Código para implementar:

```
Celula* buscaElemento(Matriz* mat, int val) {
```

```
// IMPLEMENTAR
```

```
}
```

Dica: Percorra linha por linha (via inf) e em cada linha coluna por coluna (via dir). Retorne ao encontrar.

Questão 16

Categoria: Ordenação | Linguagem: Java | Dificuldade: Médio

Implemente `public static int[] vetorOrdenado(int[] vetA, int[] vetB)` que funde dois arrays ordenados de forma decrescente retornando um terceiro em ordem crescente. Complexidade $O(m+n)$.

Código para implementar:

```
public static int[] vetorOrdenado(int[] vetA,
```

```
int[] vetB) {
```

```
// IMPLEMENTAR – fusão em  $O(m+n)$ 
```

```
}
```

Dica: Arrays estão em ordem decrescente, leia-os de trás para frente (do menor para o maior).

Questão 17

Categoria: Ordenação | Linguagem: Java | Dificuldade: Difícil

Implemente o mergeSort recursivo em Java para ordem crescente.

Código para implementar:

```
public static void mergeSort(int[] arr,
```

```
int ini, int fim) {
```

```
// IMPLEMENTAR
```

```
}
```

```
public static void merge(int[] arr,
```

```
int ini, int mid, int fim) {
```

```
// IMPLEMENTAR
```

```
}
```

Dica: Divida ao meio recursivamente até tamanho 1, depois funda os pares ordenados.

Questão 18

Categoria: Ordenação | Linguagem: Java | Dificuldade: Difícil

Implemente o quickSort em Java com particionamento de Lomuto (pivô = último elemento).

Código para implementar:

```
public static void quickSort(int[] arr,
int ini, int fim) {
// IMPLEMENTAR
}
public static int particionar(int[] arr,
int ini, int fim) {
// IMPLEMENTAR – retorna índice final do pivô
}
```

Dica: O índice i marca o limite da região menor que o pivô. Ao final, troque o pivô para a posição i+1.

Questão 19

Categoria: Ordenação | Linguagem: Java | Dificuldade: Médio

Implemente o heapSort em Java (max-heap in-place, ordem crescente).

Código para implementar:

```
public static void heapSort(int[] arr) {
// IMPLEMENTAR
}
public static void heapify(int[] arr,
int n, int i) {
// IMPLEMENTAR
}
```

Dica: Fase 1: construa o max-heap de baixo para cima. Fase 2: extraia o máximo e reheapifique.

Questão 20

Categoria: Ordenação | Linguagem: C | Dificuldade: Difícil

Implemente o radixSort em C para um array de inteiros não-negativos usando base 10.

Código para implementar:

```
void countSort(int* arr, int n, int exp);
void radixSort(int* arr, int n) {
// IMPLEMENTAR – para numeros de ate 3 digitos
}
```

Dica: Aplique counting sort para cada posição de dígito. Use (arr[i]/exp)%10 para extrair o dígito.

Questão 21

Categoria: Ordenação | Linguagem: Java | Dificuldade: Fácil

Implemente o insertionSort em Java para ordem crescente.

Código para implementar:

```
public static void insertionSort(int[] arr) {
// IMPLEMENTAR
}
```



```
}
```

Dica: Para cada elemento, desloque os maiores uma posição à direita e insira a chave na posição correta.

Questão 22

Categoria: Ordenação | Linguagem: C | Dificuldade: Médio

Implemente o selectionSort em C para ordem crescente.

Código para implementar:

```
void selectionSort(int* arr, int n) {  
    // IMPLEMENTAR  
}
```

Dica: Em cada iteração, encontre o mínimo do subarray restante e troque com a posição atual.

Questão 23

Categoria: BSP — Operações | Linguagem: C | Dificuldade: Médio

Implemente a busca recursiva em uma BSP em C: No* buscar(No* raiz, int val).

Código para implementar:

```
typedef struct No {  
    int numero;  
    struct No* esq; struct No* dir;  
} No;  
  
No* buscar(No* raiz, int val) {  
    // IMPLEMENTAR  
}
```

Dica: Se val < raiz busque à esquerda, se val > raiz busque à direita, se igual retorne o nó.

Questão 24

Categoria: BSP — Operações | Linguagem: C | Dificuldade: Médio

Implemente a inserção iterativa em BSP em C: No* inserir(No* raiz, int val).

Código para implementar:

```
No* inserir(No* raiz, int val) {  
    // IMPLEMENTAR – versao iterativa  
}
```

Dica: Desça pela árvore guardando o nó pai. Insira o novo nó como filho do pai.

Questão 25

Categoria: BSP — Operações | Linguagem: C | Dificuldade: Difícil

Implemente No* remover(No* raiz, int val) em C que remove um nó de uma BSP tratando os 3 casos: sem filhos, 1 filho e 2 filhos (use o sucessor in-ordem).

Código para implementar:

```
No* remover(No* raiz, int val) {  
    // IMPLEMENTAR – 3 casos  
}
```

Dica: 2 filhos: substitua o valor pelo sucessor in-ordem (menor da subárvore direita) e remova o sucessor.

Questão 26

Categoria: BSP — Operações | Linguagem: Java | Dificuldade: Médio

Implemente o caminhamento pós-ordem em Java para imprimir todos os valores de uma BSP.

Código para implementar:

```
void posOrdem(No cur) {  
    // IMPLEMENTAR  
}
```

Dica: Pós-ordem: esquerda -> direita -> raiz. O nó pai é processado por último.

Questão 27

Categoria: BSP — Operações | Linguagem: Java | Dificuldade: Médio

Implemente `int altura(No cur)` que retorna a altura de uma BSP em Java (folha = 0, árvore vazia = -1).

Código para implementar:

```
int altura(No cur) {  
    // IMPLEMENTAR  
}
```

Dica: Altura de um nó é 1 + maior altura entre os filhos. Árvore vazia retorna -1.

Questão 28

Categoria: BSP — Operações | Linguagem: Java | Dificuldade: Difícil

Implemente `boolean ehBSP(No cur, Integer min, Integer max)` que verifica se uma árvore binária é uma BSP válida.

Código para implementar:

```
boolean ehBSP(No cur, Integer min, Integer max) {  
    // IMPLEMENTAR  
}
```

Dica: Passe limites min e max para cada subárvore. Subárvore esquerda: todos < cur.numero, direita: todos > cur.numero.

Questão 29

Categoria: BSP — Operações | Linguagem: C | Dificuldade: Difícil

Implemente em C a função `void liberarArvore(No* raiz)` que libera toda a memória de uma BSP usando pós-ordem.

Código para implementar:

```
void liberarArvore(No* raiz) {  
    // IMPLEMENTAR  
}
```

Dica: Use pós-ordem obrigatoriamente: libere os filhos antes do pai para não perder referências.

Questão 30

Categoria: BSP — Operações | Linguagem: Java | Dificuldade: Médio

Implemente `int contarFolhas(No cur)` que conta os nós folha (sem filhos) de uma BSP.

Código para implementar:

```
int contarFolhas(No cur) {  
    // IMPLEMENTAR  
}
```

Dica: Um nó é folha se ambos `esq` e `dir` são `null`. Some as folhas de cada subárvore.

Questão 31

Categoria: Lista Dupla Encadeada | Linguagem: C | Dificuldade: Fácil

Implemente `void imprimirLista(CelulaDupla* primeiro)` para a lista duplamente encadeada circular com cabeça, imprimindo todos os elementos exceto a cabeça.

Código para implementar:

```
void imprimirLista(CelulaDupla* primeiro) {  
    // IMPLEMENTAR  
}
```

Dica: Comece em `primeiro->prox` e avance até retornar à cabeça (`cur == primeiro`).

Questão 32

Categoria: Lista Dupla Encadeada | Linguagem: C | Dificuldade: Médio

Implemente `int tamanhoLista(CelulaDupla* primeiro)` que conta os elementos da lista duplamente encadeada circular com cabeça.

Código para implementar:

```
int tamanhoLista(CelulaDupla* primeiro) {  
    // IMPLEMENTAR  
}
```

Dica: Percorra do `primeiro->prox` até voltar à cabeça. Conte cada nó.

Questão 33

Categoria: Lista Dupla Encadeada | Linguagem: C | Dificuldade: Difícil

Implemente `void inserirOrdenado(int x)` para a lista duplamente encadeada circular com cabeça, mantendo a lista em ordem crescente.

Código para implementar:

```
void inserirOrdenado(int x) {  
    // IMPLEMENTAR  
}
```

Dica: Avance enquanto `elemento atual < x`. Insira a nova célula antes de `cur` ajustando 4 ponteiros.

Questão 34

Categoria: Análise de Complexidade | Linguagem: Ambos | Dificuldade: Fácil

Analise o seguinte trecho e determine sua complexidade: `for(i=0; i<n; i++) for(j=i; j<n; j++) operacao();`

Código para implementar:

```
// Qual e a complexidade de tempo?
```

```
// Justifique contando as operacoes.
```

Dica: Some as iterações do loop interno para cada valor de i.

Questão 35

Categoria: Análise de Complexidade | Linguagem: Ambos | Dificuldade: Médio

Resolva a recorrência $T(n) = T(n-1) + O(1)$ e determine a complexidade. Dê um exemplo de algoritmo com essa recorrência.

Código para implementar:

```
// Resolva expandindo a recorrência.
```

Dica: Expanda substituindo $T(n-1)$ recursivamente até o caso base.

Questão 36

Categoria: Análise de Complexidade | Linguagem: Ambos | Dificuldade: Difícil

Aplique o Teorema Mestre à recorrência $T(n) = 4T(n/2) + O(n)$ e determine a complexidade.

Código para implementar:

```
// Identifique a, b e f(n).
```

```
// Compare f(n) com  $n^{\log_b a}$ .
```

Dica: Caso 1 do Teorema Mestre: $f(n)$ polinomialmente menor que $n^{\log_b a}$.

Questão 37

Categoria: Análise de Complexidade | Linguagem: Ambos | Dificuldade: Médio

Justifique por que o caminhamento em-ordem de uma BSP produz uma sequência ordenada crescente. Argumente formalmente.

Código para implementar:

```
// Argumento por inducao estrutural.
```

Dica: Use a propriedade BSP (esq < raiz < dir) e inducao estrutural.

Questão 38

Categoria: Ordenação | Linguagem: Java | Dificuldade: Difícil

Implemente o countingSort em Java para um array de inteiros no intervalo $[0, k]$.

Código para implementar:

```
public static int[] countingSort(int[] arr, int k) {
```

```
// IMPLEMENTAR
```

```
}
```

Dica: Fase 1: frequencias. Fase 2: prefixo acumulado. Fase 3: de tras para frente para manter estabilidade.

Questão 39

Categoria: Ordenação | Linguagem: C | Dificuldade: Médio

Implemente o bubbleSort em C com otimização de parada antecipada.

Código para implementar:

```
void bubbleSort(int* arr, int n) {  
    // IMPLEMENTAR – com flag de parada  
}
```

Dica: Se uma passagem completa não fizer nenhuma troca, o array já está ordenado.

Questão 40

Categoria: BSP — Operações | Linguagem: Java | Dificuldade: Difícil

Implemente No kEsimMenor(No cur, int k) que retorna o k-ésimo menor elemento de uma BSP (k=1 é o menor).

Código para implementar:

```
int contarNos(No cur) {  
    if (cur==null) return 0;  
    return 1+contarNos(cur.esq)+contarNos(cur.dir);  
}  
  
No kEsimMenor(No cur, int k) {  
    // IMPLEMENTAR  
}
```

Dica: O número de nós na subárvore esquerda + 1 é a posição da raiz no in-order. Compare com k.

Questão 41

Categoria: BSP — Operações | Linguagem: Java | Dificuldade: Médio

Implemente int somaFolhas(No cur) que retorna a soma dos valores de todos os nós folha de uma BSP.

Código para implementar:

```
int somaFolhas(No cur) {  
    // IMPLEMENTAR  
}
```

Dica: Folha: esq==null && dir==null. Some recursivamente as folhas de cada subárvore.

Questão 42

Categoria: Ordenação | Linguagem: Java | Dificuldade: Difícil

Implemente o mergeSort para uma lista simplesmente encadeada em Java. Retorne a nova cabeça da lista ordenada.

Código para implementar:

```
class No { int val; No prox; }  
  
No mergeSort(No head) {  
    // IMPLEMENTAR  
}  
  
No merge(No a, No b) {  
    // IMPLEMENTAR  
}
```

```
}
```

Dica: Use ponteiro lento/rapido para encontrar o meio da lista sem saber o tamanho.

Questão 43

Categoria: BSP — Operações | Linguagem: C | Dificuldade: Difícil

Implemente `No* converterArrayBSP(int* arr, int ini, int fim)` que converte um array ordenado em uma BSP balanceada (elemento do meio vira raiz).

Código para implementar:

```
No* converterArrayBSP(int* arr,
int ini, int fim) {
// IMPLEMENTAR
}
```

Dica: O elemento do meio vira a raiz. Aplique recursivamente para a metade esquerda e direita.

Questão 44

Categoria: Análise de Complexidade | Linguagem: Ambos | Dificuldade: Fácil

Determine a complexidade de busca em lista encadeada simples não ordenada (tamanho n). E em lista ordenada (com parada antecipada)?

Código para implementar:

```
// Analise os dois cenários e justifique.
```

Dica: Lista encadeada não tem acesso aleatório, portanto busca binária é impossível mesmo ordenada.

Questão 45

Categoria: Matriz Flexível | Linguagem: Java | Dificuldade: Médio

Implemente em Java o método `void imprimirMatrizListas()` na classe `Matriz` que imprime todos os elementos de todas as listas internas, linha por linha.

Código para implementar:

```
void imprimirMatrizListas() {
// IMPLEMENTAR
}
```

Dica: Dois loops: linha (inf) e coluna (prox). Para cada CelulaMat, percorra a lista interna a partir de primeiro.prox (pule a sentinela).

Questão 46

Categoria: Lista Dupla Encadeada | Linguagem: Java | Dificuldade: Difícil

Implemente `void mesclarListas(Lista l1, Lista l2)` que insere cada pilha de $l2$ em $l1$ em ordem crescente de tamanho de pilha.

Código para implementar:

```
void mesclarListas(Lista l1, Lista l2) {
// IMPLEMENTAR
}
```

Dica: Para cada CelulaLista de $l2$, calcule o tamanho e encontre a posição correta em $l1$ (insertion sort por tamanho).

Questão 47

Categoria: Lista Dupla Encadeada | Linguagem: Java | Dificuldade: Difícil

Implemente void ordenarPilhasLista(Lista l) que ordena as pilhas de uma Lista de Pilhas em ordem crescente de tamanho, usando insertion sort na lista.

Código para implementar:

```
void ordenarPilhasLista(Lista l) {  
    // IMPLEMENTAR  
}
```

Dica: Insertion sort adaptado para listas encadeadas: para cada elemento, encontre a posicao correta na parte ja ordenada.

Questão 48

Categoria: Análise de Complexidade | Linguagem: Ambos | Dificuldade: Médio

Resolva $T(n) = 2T(n/2) + O(n)$ pelo Teorema Mestre. Qual algoritmo classico tem essa recorrência?

Código para implementar:

```
// Identifique o caso do Teorema Mestre  
// e resolva.
```

*Dica: Caso 2: $f(n) = \Theta(n^{\log_b(a)}) \rightarrow T(n) = \Theta(n^{\log_b(a)} * \log n)$.*

Questão 49

Categoria: BSP — Operações | Linguagem: Java | Dificuldade: Médio

Implemente int maiorElemento(No cur) que retorna o maior valor armazenado em uma BSP (sem assumir que a árvore está balanceada).

Código para implementar:

```
int maiorElemento(No cur) {  
    // IMPLEMENTAR  
}
```

Dica: Em uma BSP, o maior elemento esta sempre no nó mais a direita (sem filho direito).

Questão 50

Categoria: BSP — Operações | Linguagem: Java | Dificuldade: Médio

Implemente void emOrdem(No cur) para imprimir os elementos de uma BSP em ordem crescente.

Código para implementar:

```
void emOrdem(No cur) {  
    // IMPLEMENTAR  
}
```

Dica: Em-ordem: esquerda -> raiz -> direita. Em BSP, produz sequencia crescente.

Gabaritos

Gabarito — Questão 1

Categoria: Lista Dupla Encadeada | Linguagem: C | Dificuldade: Médio

```
void inserirFim(int x) {  
    CelulaDupla* nova = novaCelulaDupla(x);  
    CelulaDupla* ultimo = primeiro->ant;  
    nova->prox = primeiro;  
    nova->ant = ultimo;  
    ultimo->prox = nova;  
    primeiro->ant = nova;  
}
```

Complexidade: **O(1)**

Gabarito — Questão 2

Categoria: Lista Dupla Encadeada | Linguagem: C | Dificuldade: Médio

```
void removerInicio() {  
    if (primeiro->prox == primeiro) return;  
    CelulaDupla* rem = primeiro->prox;  
    primeiro->prox = rem->prox;  
    rem->prox->ant = primeiro;  
    free(rem);  
}
```

Complexidade: **O(1)**

Gabarito — Questão 3

Categoria: Lista Dupla Encadeada | Linguagem: C | Dificuldade: Difícil

```
void inverterLista() {  
    CelulaDupla* cur = primeiro;  
    do {  
        CelulaDupla* tmp = cur->prox;  
        cur->prox = cur->ant;  
        cur->ant = tmp;  
        cur = tmp;  
    } while (cur != primeiro);  
}
```

Complexidade: **O(n)**

Gabarito — Questão 4

Categoria: Lista Dupla Encadeada | Linguagem: Java | Dificuldade: Médio

```
CelulaDupla intercalarReverso(CelulaDupla c1,  
CelulaDupla c2) {
```



```

CelulaDupla cab = new CelulaDupla();
cab.prox = cab; cab.ant = cab;
CelulaDupla p1 = c1.ant;
CelulaDupla p2 = c2.ant;
while (p1 != c1 || p2 != c2) {
    if (p1 != c1) {
        inserirFimLocal(cab, p1.elemento);
        p1 = p1.ant;
    }
    if (p2 != c2) {
        inserirFimLocal(cab, p2.elemento);
        p2 = p2.ant;
    }
}
return cab;
}

```

Complexidade: **$O(m+n)$**

Gabarito — Questão 5

Categoria: Lista Dupla Encadeada | Linguagem: Java | Dificuldade: Fácil

```

CelulaLista maiorPilha() {
    CelulaLista maior = null;
    int maxTam = -1;
    CelulaLista cur = inicio;
    while (cur != null) {
        int tam = 0;
        CelulaPilha p = cur.topo;
        while (p != null) { tam++; p = p.prox; }
        if (tam > maxTam) { maxTam = tam; maior = cur; }
        cur = cur.prox;
    }
    return maior;
}

```

Complexidade: **$O(n*k)$ onde n = número de pilhas, k = tamanho médio**

Gabarito — Questão 6

Categoria: Árvore Binária de Pesquisa | Linguagem: Java | Dificuldade: Médio

```

int contarNo(No no, char pr, char ul) {
    if (no == null) return 0;
    int cnt = 0;
    if (no.lettra == pr)
        cnt += contarNo2(no.raiz, pr, ul);
    cnt += contarNo(no.esq, pr, ul);
    cnt += contarNo(no.dir, pr, ul);
}

```

```

return cnt;
}

int contarNo2(No2 n2, char pr, char ul) {
if (n2 == null) return 0;
int cnt = 0;
String p = n2.palavra;
if (p.charAt(0)==pr &&
p.charAt(p.length()-1)==ul) cnt=1;
cnt += contarNo2(n2.esq, pr, ul);
cnt += contarNo2(n2.dir, pr, ul);
return cnt;
}

```

Complexidade: **$O(n*k)$** onde **n** = nós No, **k** = palavras por nó

Gabarito — Questão 7

Categoria: Árvore Binária de Pesquisa | Linguagem: Java | Dificuldade: Médio

```

int contarPalavras(String padrao) {
char inicial = padrao.charAt(0);
int tam = padrao.length();
No noLetra = buscarNo(raiz, inicial);
if (noLetra == null) return 0;
return contarNo2Padrao(noLetra.raiz, inicial, tam);
}

No buscarNo(No cur, char letra) {
if (cur == null) return null;
if (cur.letra == letra) return cur;
if (letra < cur.letra)
return buscarNo(cur.esq, letra);
return buscarNo(cur.dir, letra);
}

int contarNo2Padrao(No2 n2, char ini, int tam) {
if (n2 == null) return 0;
int cnt = 0;
if (n2.palavra.charAt(0)==ini &&
n2.palavra.length()==tam) cnt=1;
cnt += contarNo2Padrao(n2.esq, ini, tam);
cnt += contarNo2Padrao(n2.dir, ini, tam);
return cnt;
}

```

Complexidade: **$O(h + k)$** onde **h** = altura da árvore, **k** = palavras com aquela letra

Gabarito — Questão 8

Categoria: Árvore Binária de Pesquisa | Linguagem: Java | Dificuldade: Difícil

```

public void inserir(String palavra) {

```

```

char letra = palavra.charAt(0);
raiz = inserirNo(raiz, letra, palavra);
}

private No inserirNo(No cur, char l, String p) {
if (cur == null) {
cur = new No(); cur.letra = l;
cur.primeiro = new Celula();
cur.ultimo = cur.primeiro;
}
if (l == cur.letra) inserirNaLista(cur, p);
else if (l < cur.letra)
cur.esq = inserirNo(cur.esq, l, p);
else cur.dir = inserirNo(cur.dir, l, p);
return cur;
}

private void inserirNaLista(No no, String p) {
Celula ant = no.primeiro, cur = ant.prox;
while (cur!=null && cur.palavra.compareTo(p)<0) {
ant=cur; cur=cur.prox;
}
Celula nova = new Celula(); nova.palavra = p;
nova.prox = cur; ant.prox = nova;
if (nova.prox==null) no.ultimo = nova;
}

```

Complexidade: **O(h + k)**

Gabarito — Questão 9

Categoria: Árvore Binária de Pesquisa | Linguagem: Java | Dificuldade: Difícil

```

static No insertBST(No raiz, int val) {
if (raiz == null) {
No n = new No(); n.val = val; return n;
}
if (val < raiz.val)
raiz.esq = insertBST(raiz.esq, val);
else raiz.dir = insertBST(raiz.dir, val);
return raiz;
}

static int idx = 0;
static void inOrder(No cur, int[] res) {
if (cur == null) return;
inOrder(cur.esq, res);
res[idx++] = cur.val;
inOrder(cur.dir, res);
}

```

```

public static int[] treeSort(int[] arr) {
    No raiz = null;
    for (int v : arr) raiz = insertBST(raiz, v);
    int[] res = new int[arr.length];
    idx = 0; inOrder(raiz, res); return res;
}

```

Complexidade: **$O(n \log n)$** médio, **$O(n^2)$** pior caso

Gabarito — Questão 10

Categoria: Árvore Binária de Pesquisa | Linguagem: C | Dificuldade: Difícil

```

int buscaBSP(No* raiz, int val) {
    if (!raiz) return 0;
    if (raiz->numero == val) return 1;
    if (val < raiz->numero)
        return buscaBSP(raiz->esq, val);
    return buscaBSP(raiz->dir, val);
}

Celula* inserirDecr(Celula* lista, int val) {
    Celula* nova = malloc(sizeof(Celula));
    nova->numero = val;
    if (!lista || val > lista->numero) {
        nova->prox = lista; return nova;
    }
    Celula* cur = lista;
    while (cur->prox &&
        cur->prox->numero >= val)
        cur = cur->prox;
    nova->prox = cur->prox;
    cur->prox = nova; return lista;
}

Celula* encontrarRepetidos(No* raiz,
    CelulaMatriz* inicio) {
    Celula* res = NULL;
    CelulaMatriz* linha = inicio;
    while (linha) {
        CelulaMatriz* cel = linha;
        while (cel) {
            if (buscaBSP(raiz, cel->numero))
                res = inserirDecr(res, cel->numero);
            cel = cel->prox;
        }
        linha = linha->inf;
    }
    return res;
}

```

```
}
```

Complexidade: $O(L \cdot C \cdot h)$ onde L, C = dimensões da matriz, h = altura da BSP

Gabarito — Questão 11

Categoria: Matriz Flexível | Linguagem: C | Dificuldade: Médio

```
void removerUltimaColuna(Matriz* matriz) {
    if (!matriz->inicio || !matriz->colunas) return;
    Celula* cur = matriz->inicio;
    while (cur) {
        Celula* linha = cur;
        while (linha->dir) linha = linha->dir;
        Celula* rem = linha;
        if (rem->esq) rem->esq->dir = NULL;
        free(rem);
        cur = cur->inf;
    }
    matriz->colunas--;
}
```

Complexidade: $O(L)$ onde L = número de linhas

Gabarito — Questão 12

Categoria: Matriz Flexível | Linguagem: C | Dificuldade: Difícil

```
void inserirLinha(Matriz* mat, int* valores) {
    Celula* ant = NULL, *primNova = NULL;
    Celula* ultLinha = mat->inicio;
    if (ultLinha)
        while (ultLinha->inf) ultLinha = ultLinha->inf;
    Celula* colAcima = ultLinha;
    for (int c = 0; c < mat->colunas; c++) {
        Celula* nova = calloc(1, sizeof(Celula));
        nova->elemento = valores[c];
        nova->esq = ant;
        if (ant) ant->dir = nova;
        else primNova = nova;
        if (colAcima) {
            nova->sup = colAcima;
            colAcima->inf = nova;
            colAcima = colAcima->dir;
        }
        ant = nova;
    }
    if (!mat->inicio) mat->inicio = primNova;
    mat->linhas++;
}
```

Complexidade: **O(C)** onde **C** = número de colunas

Gabarito — Questão 13

Categoria: Matriz Flexível | Linguagem: Java | Dificuldade: Médio

```
void removerImpares() {
    CelulaMat linha = inicio;
    while (linha != null) {
        CelulaMat cel = linha;
        while (cel != null) {
            Celula ant = cel.primeiro;
            Celula cur = ant.prox;
            while (cur != null) {
                if (cur.numero % 2 != 0) {
                    ant.prox = cur.prox;
                    if (cur.prox==null)
                        cel.ultimo = ant;
                } else { ant = cur; }
                cur = ant.prox;
            }
            cel = cel.prox;
        }
        linha = linha.inf;
    }
}
```

Complexidade: **O(M*N*k)**

Gabarito — Questão 14

Categoria: Matriz Flexível | Linguagem: Java | Dificuldade: Difícil

```
void removerPrimeiraColunaMatriz() {
    if (inicio == null) return;
    CelulaMat cur = inicio;
    CelulaMat novoInicio = null;
    while (cur != null) {
        CelulaMat proxLinha = cur.inf;
        CelulaMat segunda = cur.prox;
        if (segunda != null) {
            segunda.ant = null;
            if (novoInicio == null)
                novoInicio = segunda;
        }
        cur = proxLinha;
    }
    inicio = novoInicio;
}
```

Complexidade: $O(L)$

Gabarito — Questão 15

Categoria: Matriz Flexível | Linguagem: C | Dificuldade: Médio

```
Celula* buscaElemento(Matriz* mat, int val) {
Celula* linha = mat->inicio;
while (linha) {
Celula* col = linha;
while (col) {
if (col->elemento == val) return col;
col = col->dir;
}
linha = linha->inf;
}
return NULL;
}
```

Complexidade: $O(L \cdot C)$

Gabarito — Questão 16

Categoria: Ordenação | Linguagem: Java | Dificuldade: Médio

```
public static int[] vetorOrdenado(int[] vetA,
int[] vetB) {
int m=vetA.length, n=vetB.length;
int[] res = new int[m+n];
int i=m-1, j=n-1, k=0;
while (i>=0 && j>=0) {
if (vetA[i] <= vetB[j]) res[k++]=vetA[i--];
else res[k++]=vetB[j--];
}
while (i>=0) res[k++]=vetA[i--];
while (j>=0) res[k++]=vetB[j--];
return res;
}
```

Complexidade: $O(m+n)$

Gabarito — Questão 17

Categoria: Ordenação | Linguagem: Java | Dificuldade: Difícil

```
public static void mergeSort(int[] arr,
int ini, int fim) {
if (ini >= fim) return;
int mid = (ini+fim)/2;
mergeSort(arr, ini, mid);
mergeSort(arr, mid+1, fim);
}
```

```

merge(arr, ini, mid, fim);
}

public static void merge(int[] arr,
int ini, int mid, int fim) {
int[] tmp = new int[fim-ini+1];
int i=ini, j=mid+1, k=0;
while (i<=mid && j<=fim) {
if (arr[i]<=arr[j]) tmp[k++]=arr[i++];
else tmp[k++]=arr[j++];
}
while (i<=mid) tmp[k++]=arr[i++];
while (j<=fim) tmp[k++]=arr[j++];
for (int l=0; l<tmp.length; l++)
arr[ini+l]=tmp[l];
}

```

Complexidade: **$O(n \log n)$** em todos os casos. **$O(n)$** de espaço extra.

Gabarito — Questão 18

Categoria: Ordenação | Linguagem: Java | Dificuldade: Difícil

```

public static void quickSort(int[] arr,
int ini, int fim) {
if (ini >= fim) return;
int p = particionar(arr, ini, fim);
quickSort(arr, ini, p-1);
quickSort(arr, p+1, fim);
}

public static int particionar(int[] arr,
int ini, int fim) {
int pivo = arr[fim], i = ini-1;
for (int j=ini; j<fim; j++) {
if (arr[j] <= pivo) {
i++;
int tmp=arr[i]; arr[i]=arr[j]; arr[j]=tmp;
}
}
int tmp=arr[i+1];
arr[i+1]=arr[fim]; arr[fim]=tmp;
return i+1;
}

```

Complexidade: **$O(n \log n)$** médio, **$O(n^2)$** pior caso

Gabarito — Questão 19

Categoria: Ordenação | Linguagem: Java | Dificuldade: Médio

```

public static void heapSort(int[] arr) {

```



```

int n = arr.length;
for (int i=n/2-1; i>=0; i--)
    heapify(arr,n,i);
for (int i=n-1; i>0; i--) {
    int tmp=arr[0]; arr[0]=arr[i]; arr[i]=tmp;
    heapify(arr,i,0);
}
}

public static void heapify(int[] arr,
int n, int i) {
    int maior=i, e=2*i+1, d=2*i+2;
    if (e<n && arr[e]>arr[maior]) maior=e;
    if (d<n && arr[d]>arr[maior]) maior=d;
    if (maior != i) {
        int tmp=arr[i]; arr[i]=arr[maior];
        arr[maior]=tmp; heapify(arr,n,maior);
    }
}
}

```

Complexidade: **$O(n \log n)$** em todos os casos. **$O(1)$** de espaço extra.

Gabarito — Questão 20

Categoria: Ordenação | Linguagem: C | Dificuldade: Difícil

```

void countSort(int* arr, int n, int exp) {
    int* saida = malloc(n * sizeof(int));
    int count[10] = {0};
    for (int i=0; i<n; i++)
        count[(arr[i]/exp)%10]++;
    for (int i=1; i<10; i++)
        count[i] += count[i-1];
    for (int i=n-1; i>=0; i--)
        saida[--count[(arr[i]/exp)%10]] = arr[i];
    for (int i=0; i<n; i++) arr[i] = saida[i];
    free(saida);
}

void radixSort(int* arr, int n) {
    for (int exp=1; exp<=100; exp*=10)
        countSort(arr, n, exp);
}

```

Complexidade: **$O(d*(n+k))$** onde **d = dígitos**, **k = 10**. Para **d constante**: **$O(n)$** .

Gabarito — Questão 21

Categoria: Ordenação | Linguagem: Java | Dificuldade: Fácil

```

public static void insertionSort(int[] arr) {
    for (int i=1; i<arr.length; i++) {

```

```

int chave = arr[i];
int j = i-1;
while (j>=0 && arr[j]>chave) {
arr[j+1] = arr[j]; j--;
}
arr[j+1] = chave;
}
}

```

Complexidade: **$O(n^2)$ pior caso, $O(n)$ para array quase ordenado**

Gabarito — Questão 22

Categoria: Ordenação | Linguagem: C | Dificuldade: Médio

```

void selectionSort(int* arr, int n) {
for (int i=0; i<n-1; i++) {
int minIdx = i;
for (int j=i+1; j<n; j++)
if (arr[j] < arr[minIdx]) minIdx = j;
int tmp=arr[i];
arr[i]=arr[minIdx]; arr[minIdx]=tmp;
}
}

```

Complexidade: **$O(n^2)$ em todos os casos. Faz no máximo $n-1$ trocas.**

Gabarito — Questão 23

Categoria: BSP — Operações | Linguagem: C | Dificuldade: Médio

```

No* buscar(No* raiz, int val) {
if (raiz == NULL) return NULL;
if (raiz->numero == val) return raiz;
if (val < raiz->numero)
return buscar(raiz->esq, val);
return buscar(raiz->dir, val);
}

```

Complexidade: **$O(h)$: $O(\log n)$ balanceada, $O(n)$ degenerada**

Gabarito — Questão 24

Categoria: BSP — Operações | Linguagem: C | Dificuldade: Médio

```

No* inserir(No* raiz, int val) {
No* novo = malloc(sizeof(No));
novo->numero=val; novo->esq=novo->dir=NULL;
if (!raiz) return novo;
No* cur=raiz, *pai=NULL;
while (cur) {
pai = cur;

```

```

if (val < cur->numero) cur=cur->esq;
else cur=cur->dir;
}
if (val < pai->numero) pai->esq = novo;
else pai->dir = novo;
return raiz;
}

```

Complexidade: **O(h)**

Gabarito — Questão 25

Categoria: BSP — Operações | Linguagem: C | Dificuldade: Difícil

```

No* menorNo(No* cur) {
while (cur->esq) cur=cur->esq;
return cur;
}

No* remover(No* raiz, int val) {
if (!raiz) return NULL;
if (val < raiz->numero)
raiz->esq = remover(raiz->esq, val);
else if (val > raiz->numero)
raiz->dir = remover(raiz->dir, val);
else {
if (!raiz->esq) {
No* tmp=raiz->dir; free(raiz); return tmp;
} else if (!raiz->dir) {
No* tmp=raiz->esq; free(raiz); return tmp;
} else {
No* suc = menorNo(raiz->dir);
raiz->numero = suc->numero;
raiz->dir = remover(raiz->dir,
suc->numero);
}
}
return raiz;
}

```

Complexidade: **O(h)**

Gabarito — Questão 26

Categoria: BSP — Operações | Linguagem: Java | Dificuldade: Médio

```

void posOrdem(No cur) {
if (cur == null) return;
posOrdem(cur.esq);
posOrdem(cur.dir);
System.out.print(cur.numero + " ");
}

```

```
}
```

Complexidade: **O(n)**

Gabarito — Questão 27

Categoria: BSP — Operações | Linguagem: Java | Dificuldade: Médio

```
int altura(No cur) {  
    if (cur == null) return -1;  
    int altEsq = altura(cur.esq);  
    int altDir = altura(cur.dir);  
    return 1 + Math.max(altEsq, altDir);  
}
```

Complexidade: **O(n)**

Gabarito — Questão 28

Categoria: BSP — Operações | Linguagem: Java | Dificuldade: Difícil

```
boolean ehBSP(No cur, Integer min, Integer max) {  
    if (cur == null) return true;  
    if (min!=null && cur.numero<=min) return false;  
    if (max!=null && cur.numero>=max) return false;  
    return ehBSP(cur.esq, min, cur.numero) &&  
        ehBSP(cur.dir, cur.numero, max);  
}
```

Complexidade: **O(n)**

Gabarito — Questão 29

Categoria: BSP — Operações | Linguagem: C | Dificuldade: Difícil

```
void liberarArvore(No* raiz) {  
    if (raiz == NULL) return;  
    liberarArvore(raiz->esq);  
    liberarArvore(raiz->dir);  
    free(raiz);  
}
```

Complexidade: **O(n)**

Gabarito — Questão 30

Categoria: BSP — Operações | Linguagem: Java | Dificuldade: Médio

```
int contarFolhas(No cur) {  
    if (cur == null) return 0;  
    if (cur.esq==null && cur.dir==null) return 1;  
    return contarFolhas(cur.esq)  
        + contarFolhas(cur.dir);  
}
```

Complexidade: **O(n)**

Gabarito — Questão 31

Categoria: Lista Dupla Encadeada | Linguagem: C | Dificuldade: Fácil

```
void imprimirLista(CelulaDupla* primeiro) {  
    CelulaDupla* cur = primeiro->prox;  
    while (cur != primeiro) {  
        printf("%d ", cur->elemento);  
        cur = cur->prox;  
    }  
    printf("\n");  
}
```

Complexidade: $O(n)$

Gabarito — Questão 32

Categoria: Lista Dupla Encadeada | Linguagem: C | Dificuldade: Médio

```
int tamanhoLista(CelulaDupla* primeiro) {  
    int cnt = 0;  
    CelulaDupla* cur = primeiro->prox;  
    while (cur != primeiro) {  
        cnt++; cur = cur->prox;  
    }  
    return cnt;  
}
```

Complexidade: $O(n)$

Gabarito — Questão 33

Categoria: Lista Dupla Encadeada | Linguagem: C | Dificuldade: Difícil

```
void inserirOrdenado(int x) {  
    CelulaDupla* nova = novaCelulaDupla(x);  
    CelulaDupla* cur = primeiro->prox;  
    while (cur!=primeiro && cur->elemento < x)  
        cur = cur->prox;  
    nova->prox = cur;  
    nova->ant = cur->ant;  
    cur->ant->prox = nova;  
    cur->ant = nova;  
}
```

Complexidade: $O(n)$ para encontrar posição + $O(1)$ para inserir

Gabarito — Questão 34

Categoria: Análise de Complexidade | Linguagem: Ambos | Dificuldade: Fácil

```
// i=0: n iteracoes de j  
// i=1: n-1 iteracoes
```

```
// ...
// i=n-1: 1 iteracao
// Total = n + (n-1) + ... + 1 = n(n+1)/2
// Complexidade:  $O(n^2)$ 
```

Complexidade: **$O(n^2)$** — soma aritmetica de 1 a n

Gabarito — Questão 35

Categoria: Análise de Complexidade | Linguagem: Ambos | Dificuldade: Médio

```
// T(n) = T(n-1) + c
// = T(n-2) + 2c = ... = T(1) + (n-1)c
// T(n) =  $O(n)$ 
//
// Exemplo: busca linear em array nao ordenado.
// Cada chamada reduz o problema em 1 e faz  $O(1)$ .
```

Complexidade: **$O(n)$**

Gabarito — Questão 36

Categoria: Análise de Complexidade | Linguagem: Ambos | Dificuldade: Difícil

```
// a=4, b=2, f(n)= $O(n)$ 
// log_b(a) = log_2(4) = 2
// Compare f(n)=n com  $n^2$ :
// f(n) =  $O(n^{2-e})$  para e=1 -> Caso 1
// Resultado: T(n) = Theta( $n^{\log_b(a)}$ ) = Theta( $n^2$ )
//
// Exemplo: multiplicacao de matrizes  $O(n^3)$ 
// divide em 4 subproblemas de tamanho n/2
```

Complexidade: **Theta(n^2)**

Gabarito — Questão 37

Categoria: Análise de Complexidade | Linguagem: Ambos | Dificuldade: Médio

```
// Definicao de BSP:
// esq < raiz < dir para todo no
//
// Em-ordem: esq -> raiz -> dir
//
// Base: subarvore vazia -> sequencia vazia (ok).
//
// Hipotese: recursao em esq e dir produz
// sequencias ordenadas (arvores menores).
//
// Como todos esq < raiz < todos dir,
// concatenar [seq_esq, raiz, seq_dir]
// produz sequencia crescente. QED
```

Complexidade: **Prova por inducao estrutural**

Gabarito — Questão 38

Categoria: Ordenação | Linguagem: Java | Dificuldade: Difícil

```
public static int[] countingSort(int[] arr, int k) {
    int[] count = new int[k+1];
    for (int v : arr) count[v]++;
    for (int i=1; i<=k; i++) count[i]+=count[i-1];
    int[] saida = new int[arr.length];
    for (int i=arr.length-1; i>=0; i--)
        saida[--count[arr[i]]] = arr[i];
    return saida;
}
```

Complexidade: **$O(n+k)$ tempo e espaco**

Gabarito — Questão 39

Categoria: Ordenação | Linguagem: C | Dificuldade: Médio

```
void bubbleSort(int* arr, int n) {
    for (int i=0; i<n-1; i++) {
        int trocou = 0;
        for (int j=0; j<n-1-i; j++) {
            if (arr[j] > arr[j+1]) {
                int tmp=arr[j];
                arr[j]=arr[j+1]; arr[j+1]=tmp;
                trocou = 1;
            }
        }
        if (!trocou) break;
    }
}
```

Complexidade: **$O(n^2)$ pior caso, $O(n)$ melhor caso**

Gabarito — Questão 40

Categoria: BSP — Operações | Linguagem: Java | Dificuldade: Difícil

```
No kEsimoMenor(Node cur, int k) {
    if (cur == null) return null;
    int esqCount = contarNos(cur.esq);
    if (k == esqCount+1) return cur;
    if (k <= esqCount)
        return kEsimoMenor(cur.esq, k);
    return kEsimoMenor(cur.dir,
        k-esqCount-1);
}
```

Complexidade: $O(h*n)$ sem campo size, $O(h)$ com campo size armazenado

Gabarito — Questão 41

Categoria: BSP — Operações | Linguagem: Java | Dificuldade: Médio

```
int somaFolhas(No cur) {  
    if (cur == null) return 0;  
    if (cur.esq==null && cur.dir==null)  
        return cur.numero;  
    return somaFolhas(cur.esq)  
    + somaFolhas(cur.dir);  
}
```

Complexidade: $O(n)$

Gabarito — Questão 42

Categoria: Ordenação | Linguagem: Java | Dificuldade: Difícil

```
No mergeSort(No head) {  
    if (head==null || head.prox==null) return head;  
    No lento=head, rapido=head.prox;  
    while (rapido!=null && rapido.prox!=null) {  
        lento=lento.prox;  
        rapido=rapido.prox.prox;  
    }  
    No metade = lento.prox;  
    lento.prox = null;  
    No esq = mergeSort(head);  
    No dir = mergeSort(metade);  
    return merge(esq, dir);  
}  
  
No merge(No a, No b) {  
    No dummy=new No(), cur=dummy;  
    while (a!=null && b!=null) {  
        if (a.val<=b.val){cur.prox=a;a=a.prox;}  
        else {cur.prox=b; b=b.prox;}  
        cur=cur.prox;  
    }  
    cur.prox = (a!=null) ? a : b;  
    return dummy.prox;  
}
```

Complexidade: $O(n \log n)$ tempo, $O(\log n)$ pilha

Gabarito — Questão 43

Categoria: BSP — Operações | Linguagem: C | Dificuldade: Difícil

```
No* converterArrayBSP(int* arr,
```



```

int ini, int fim) {
if (ini > fim) return NULL;
int mid = (ini+fim)/2;
No* raiz = malloc(sizeof(No));
raiz->numero = arr[mid];
raiz->esq = converterArrayBSP(arr, ini, mid-1);
raiz->dir = converterArrayBSP(arr, mid+1, fim);
return raiz;
}

```

Complexidade: **O(n)**

Gabarito — Questão 44

Categoria: Análise de Complexidade | Linguagem: Ambos | Dificuldade: Fácil

```

// Lista nao ordenada:
// Pior caso: elemento nao existe -> O(n)
// Melhor caso: primeiro elemento -> O(1)
// Medio: O(n)

// Lista ordenada:
// Ainda O(n) no pior caso.
// Vantagem: confirmar ausencia antes do fim
// (parada antecipada quando cur.val > buscado).
// Lista encadeada nao suporta busca binaria!

```

Complexidade: **O(n) em ambos os casos no pior cenario**

Gabarito — Questão 45

Categoria: Matriz Flexível | Linguagem: Java | Dificuldade: Médio

```

void imprimirMatrizListas() {
CelulaMat linha = inicio;
while (linha != null) {
CelulaMat cel = linha;
while (cel != null) {
System.out.print("[");
Celula c = cel.primeiro.prox;
while (c != null) {
System.out.print(c.numero);
if (c.prox!=null) System.out.print(",");
c = c.prox;
}
System.out.print("] ");
cel = cel.prox;
}
System.out.println();
linha = linha.inf;
}

```

```
}  
}
```

Complexidade: $O(M \cdot N \cdot k)$

Gabarito — Questão 46

Categoria: Lista Dupla Encadeada | Linguagem: Java | Dificuldade: Difícil

```
int tamPilha(CelulaLista cl) {  
    int t=0; CelulaPilha p=cl.topo;  
    while (p!=null){t++;p=p.prox;} return t;  
}  
  
void mesclarListas(Lista l1, Lista l2) {  
    CelulaLista cur2 = l2.inicio;  
    while (cur2 != null) {  
        CelulaLista prox2 = cur2.prox;  
        int tam2 = tamPilha(cur2);  
        CelulaLista ant=null, cur1=l1.inicio;  
        while (cur1!=null && tamPilha(cur1)<=tam2){  
            ant=cur1; cur1=cur1.prox;  
        }  
        cur2.prox = cur1;  
        if (ant==null) l1.inicio = cur2;  
        else ant.prox = cur2;  
        if (cur1==null) l1.fim = cur2;  
        cur2 = prox2;  
    }  
}
```

Complexidade: $O(n \cdot m \cdot k)$

Gabarito — Questão 47

Categoria: Lista Dupla Encadeada | Linguagem: Java | Dificuldade: Difícil

```
void ordenarPilhasLista(Lista l) {  
    if (l.inicio==null) return;  
    CelulaLista sorted = null;  
    CelulaLista cur = l.inicio;  
    while (cur != null) {  
        CelulaLista prox = cur.prox;  
        int tamCur = tamPilha(cur);  
        if (sorted==null ||  
            tamPilha(sorted)>=tamCur) {  
            cur.prox=sorted; sorted=cur;  
        } else {  
            CelulaLista s = sorted;  
            while (s.prox!=null &&  
                tamPilha(s.prox)<tamCur)
```

```

s=s.prox;
cur.prox=s.prox; s.prox=cur;
}
cur = prox;
}
l.inicio = sorted;
}

```

Complexidade: $O(n^2 \cdot k)$

Gabarito — Questão 48

Categoria: Análise de Complexidade | Linguagem: Ambos | Dificuldade: Médio

```

// a=2, b=2, f(n)=O(n)
// log_b(a) = log_2(2) = 1
// Compare f(n)=n com n^1=n:
// f(n) = Theta(n^1) -> Caso 2
// T(n) = Theta(n log n)
//
// Algoritmo: Merge Sort
// Divide em 2 subproblemas de tamanho n/2
// e faz O(n) de trabalho para fundir.

```

Complexidade: $\Theta(n \log n)$

Gabarito — Questão 49

Categoria: BSP — Operações | Linguagem: Java | Dificuldade: Médio

```

int maiorElemento(No cur) {
    if (cur == null)
        throw new RuntimeException("Arvore vazia");
    while (cur.dir != null) cur = cur.dir;
    return cur.numero;
}

```

Complexidade: $O(h)$

Gabarito — Questão 50

Categoria: BSP — Operações | Linguagem: Java | Dificuldade: Médio

```

void emOrdem(No cur) {
    if (cur == null) return;
    emOrdem(cur.esq);
    System.out.print(cur.numero + " ");
    emOrdem(cur.dir);
}

```

Complexidade: $O(n)$
